ProveML: Inline Claim Markup for Deterministic Verification of AI-Generated Text

Technical Report

Shane Deconinck
abovebeyond.ai
hello@shanedeconinck.be

August 2026

Abstract

Organizations are deploying LLMs in consequential settings under regulatory pressure, but most generated text carries no machine-checkable link between individual claims and the underlying data. Each generation of models produces more fluent output, but none can reliably signal its own uncertainty.

We present ProveML, a lightweight Markdown extension that places AI-generated claims inside a verifiable boundary. Three inline constructs declare entities, link facts to a data source, and check qualitative judgments against composable, pre-declared thresholds:

```
@[student:42]{Alex} scored %[passRate]{5}% and is ?[r:  IS_AT_RISK]{at risk}.
```

Verification is deterministic — string equality and arithmetic against a flat key-value store, with no model in the loop — so it is instant (under 0.2 ms per claim), explainable, and can sit inside a generation loop that feeds each error back to the model; the same verifier can audit text it did not generate.

What it requires, costs and cannot do. ProveML applies where the claims are backed by an addressable structured source: without a fact store there is nothing to resolve against. Marked-up responses ran 52–60% longer in characters than the same text without markup, and three quarters of queries needed no correction pass while a quarter exhausted all three. Exact string equality forbids rounded prose — a verified sentence must say `391035000000`, not "$391 billion" — derived values with no path in the store cannot be bound, and the threshold construct, while specified and tested, was never produced by a model in our benchmark runs.

We evaluate across four models (3.8B to API-scale) and two domains — a generated educational benchmark and real SEC EDGAR filings — with a reimplemented SymGen baseline and ablations on prompt language and context size. Three findings carry the paper. Whether a model produces verifiable markup at all depends on the shape of the request more than on the size of the model. Verification rate alone overstates how much of a report has been checked, so we pair it with a markup-coverage metric and show the two can diverge widely at identical rates. And both ProveML and substitution fail on addressability, but differently: substitution leaves a hole in the sentence, ProveML leaves a flagged claim carrying the value that was expected instead. The specification, verifier, reference implementation, benchmarks and all run artifacts are published.

Keywords: AI verification, markup language, deterministic verification, structured data, threshold registry, hallucination detection, inline tagging

1 Introduction

Large Language Models (LLMs) produce fluent, contextually appropriate text that is increasingly difficult to distinguish from human-authored content. This fluency, however, coexists with a fundamental reliability problem: **LLMs hallucinate**. They generate text that is plausible but factually incorrect, including fabricated citations, invented statistics, and misattributed claims.

In one line: **ProveML is iXBRL for AI-generated text.** Financial reporting solved a version of this problem two decades ago by embedding machine-readable tags in human-readable filings, so a regulator can audit a number without reading the prose around it (XBRL International, 2013). ProveML applies that idea where the author is a model rather than a person, and adds what that setting needs: entity scoping, so a value is bound to the record it describes, and a threshold registry, so a qualitative judgment is a declared predicate rather than a choice of adjective.

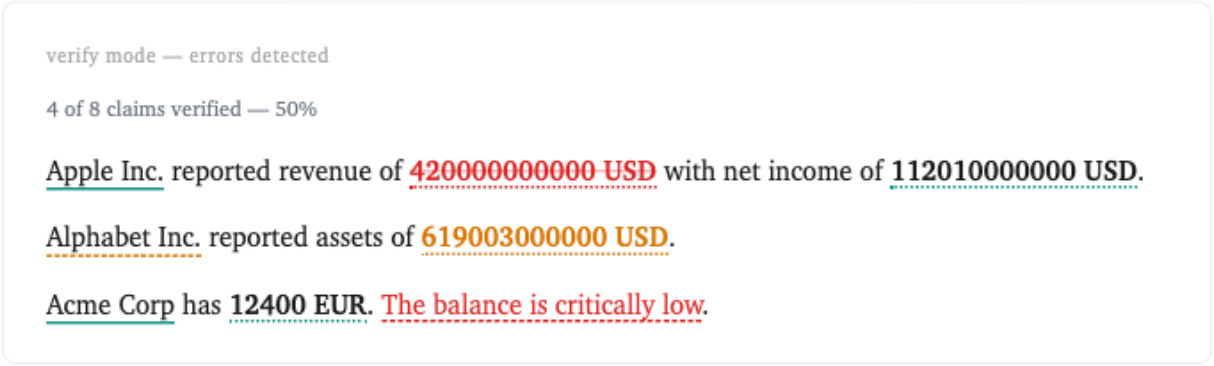

Figure 1: What the reader sees. The same marked-up text after verification: a wrong revenue figure struck through in red, an entity that does not exist in the store in amber, and a judgment whose registered condition does not hold in dashed red. Nothing here required reading the markup, and nothing required a model. Section 6 covers the rendering states; Figure 5 shows the same text with the fact-store path behind each claim made visible.

Two limits belong here rather than at the end, because they decide whether the rest is relevant to a given reader. First, exact string equality means a verified sentence states canonical values: a report can say `391035000000` but not "$391 billion", which rules out the rounded phrasing that financial and journalistic prose normally uses. Second, the threshold construct is specified, tested and demonstrated below, but no model in our benchmark runs produced one, so every rate we report measures entity and fact markup only. Section 11.5 lists the rest.

Recent theoretical work suggests that **hallucination is not merely an incidental bug but reflects structural properties** of how language models operate. Kalai and Vempala (2024) prove formally that any language model satisfying natural calibration conditions *must* hallucinate, at a rate approaching the fraction of facts that appear exactly once in the training data; subsequent work by Kalai et al. (2025) shows that standard training and evaluation pipelines actively reward guessing over acknowledging uncertainty. Xu et al. (2024) use computability theory to show that LLMs cannot learn all computable functions and will therefore inevitably produce confabulations when used as general-purpose reasoners. More broadly, hallucination is a well-documented reliability problem across deep-learning-based natural language generation systems (Ji et al., 2023).

Current mitigations reduce hallucination rates but cannot eliminate them. RLHF improves instruction-following and truthfulness (Ouyang et al., 2022). Even so, aligned models may still produce confident-sounding but incorrect responses. RAG grounds responses in documents but models can still misrepresent retrieved context; Magesh et al. (2025) found that the

leading legal research tools — Lexis+ AI and Thomson Reuters' Westlaw AI-Assisted Research and Ask Practical Law AI — each hallucinate between 17% and 33% of the time, despite vendor claims of near-elimination. Emerging interpretability research can detect some internal uncertainty signals (Orgad et al., 2025; Farquhar et al., 2024), but probes do not generalize across tasks, entropy methods fail on confident errors, and probing methods require white-box access. McCain et al. (2026) report that roughly 73% of agent tool calls on a major API still appear to retain a human in the loop.

Meanwhile, the consequences are real and adoption is accelerating. In 2025, Deloitte delivered a government report with AI-fabricated citations (Paoli, 2025). Earlier, fabricated ChatGPT citations were submitted to a U.S. federal court (U.S. District Court, S.D.N.Y., 2023), and a factual error in the launch demo materials for Google's Bard contributed to a $100 billion market-cap drop (Coulter and Bensinger, 2023). Yet the majority of organizations now use AI in at least one function (McKinsey & Company, 2025), and Gartner projected that over 80% of enterprises would have used generative AI APIs or deployed generative AI-enabled applications by 2026 (Gartner, 2023). **Organizations are not waiting for hallucination to be solved**; they are deploying now and managing the risk.

This creates precisely the conditions described in Bainbridge's classic analysis of the **ironies of automation** (Bainbridge, 1983) and well-documented in the trust-in-automation literature (Lee and See, 2004; Parasuraman and Riley, 1997): as automated systems become more reliable, human operators trust them more and monitor them less. When the rare failure occurs, it is more likely to go undetected precisely because the operator has become complacent. In the context of AI-generated text, this dynamic is particularly acute. An LLM that produces correct output 97% of the time creates a more dangerous situation than one that produces correct output 80% of the time, because the 97% system induces trust that suppresses human verification behavior.

The regulatory landscape compounds the problem. The EU AI Act (European Parliament and Council of the European Union, 2024) classifies AI systems used for consequential decisions as "high-risk" (Annex III), requiring transparency, human oversight, and risk management. The 2026 Digital Omnibus (European Parliament and Council of the European Union, 2026) deferred those stand-alone high-risk obligations to 2 December 2027 (and to 2 August 2028 for AI embedded in regulated products) while leaving the Article 50 transparency duties in force from 2 August 2026 — a deferral that buys providers and deployers time to build verifiable-claim infrastructure rather than removing the need for it. Yet to our knowledge, **no existing framework combines inline claim markup, deterministic mismatch detection against structured data, and composable threshold inference** in a single system. The closest recent work binds claims to evidence contracts (Xu et al., 2026), emits provenance triples (Wei et al., 2026), or maintains typed key-value state with non-probabilistic admission (Liu et al., 2026), but none extends an authoring format with inline claim markup, and none constrains qualitative wording through a declared threshold vocabulary.[1]

Current approaches to the AI reliability problem include several strategies, none of which addresses the structural verification gap:

1. **Post-hoc fact-checking** (FActScore (Min et al., 2023), SAFE (Wei et al., 2024), FacTool (Chern et al., 2023), SelfCheckGPT (Manakul et al., 2023), RefChecker (Hu et al., 2024)): decompose, sample, or re-check generated text after the fact against external sources or model-generated consistency signals. These systems use non-deterministic checks in the

[1]We screened the titles and abstracts of the 4,617 papers in the ACL 2026 main, short, findings and industry volumes against concept patterns for claim-level attribution, structured-data faithfulness, symbolic or deterministic verification, provenance and markup schemes, then read the resulting candidates by hand. Title-and-abstract screening can miss a mechanism described only in a paper's body, and the sweep itself was not preserved as a runnable artifact (unlike the evaluation, which regenerates from stored runs), so this is the result of a search rather than an exhaustive negative.

verification loop, making verification itself probabilistic and subject to many of the same failure modes as the generation process. Domain-specific pipelines such as FinGround (Guo et al., 2026) add deterministic sub-steps (recomputing a stated figure from a table) but still decompose and classify claims with a model.

2. **Output validation** (Guardrails AI (Guardrails AI, 2024), DSPy (Khattab et al., 2024), NeMo Guardrails (Rebedea et al., 2023)): programmatic constraint enforcement that can operate at output or sentence level. Guardrails AI's provenance validators support sentence-level checks against source text; DSPy compiles declarative LM pipelines into optimized prompts. However, verification in these systems typically relies on LLM judgments or embedding similarity; where programmatic checks exist (DSPy's assertions), they are not claim-level checks against addressable paths in structured data stores.

3. **Attribution** (RARR (Gao et al., 2023a), WebGPT (Nakano et al., 2021), OpenAI response annotations): link claims to source documents or passages. Some systems (e.g., Google's grounding API (Google, 2024)) additionally return support scores and claim-to-source metadata. These track *where* a claim came from, not *whether the claimed value is correct*. Verification, where present, is probabilistic and against unstructured text.

None of these provide a structural guarantee that every factual claim in AI-generated text is an addressable, deterministically verifiable reference to a source record in a structured data store.

We present ProveML (Provable Markup Language), a system that fills this gap. (The name is distinct from the earlier DARPA-era Proof/Provenance Markup Language, PML (Pinheiro da Silva et al., 2006), an interlingua for representing proof and provenance metadata; ProveML shares neither its lineage nor its mechanism.) ProveML is an inline claim markup language embedded in AI-generated text where:

- Every entity reference is verified against a structured data store (name match).
- Every factual claim resolves to a specific addressable path in the data store (value match).
- Every inference is checked against a threshold registry defined outside the generation process.
- Verification is fully deterministic, with no model in the verification loop.
- Unverified or mismatched claims are structurally detectable and can be flagged by the rendering layer.

The closest structural analog is iXBRL (Inline XBRL) (XBRL International, 2013), which embeds machine-readable tags in human-readable financial reports, enabling automated audit of reported figures. ProveML extends this concept from human-authored financial text to AI-generated text across any domain, adding inference chains, entity scoping, and threshold registries (see Figure 6 for the architectural distinction).

The key insight underlying ProveML is that *verification should be a structural property of the generated text, not a post-hoc evaluation.* Just as a type system prevents certain classes of errors at compile time rather than detecting them at runtime, ProveML makes certain classes of AI errors (factual inaccuracy, entity misattribution, unjustified inference) structurally detectable at generation time.

Design philosophy. ProveML is deliberately assembled from established patterns rather than invented from scratch:

- Host language: Markdown (natively produced by LLMs)
- Inline tagging model from iXBRL (XBRL International, 2013)
- Operator vocabulary from FHIR clinical reference ranges (HL7 International, 2023) and JSON Schema validation
- Fact store from the Entity-Attribute-Value (EAV) pattern; Liu et al. (2026) arrive at a comparable typed key-value state with a non-probabilistic admission check, in the agent/tool setting rather than in text
- Verify-then-render pipeline from standard compiler design

A skeptical reader should find each component familiar. What ProveML contributes is the *specific combination* applied to AI-generated text, together with two mechanisms we have not seen elsewhere: the threshold registry as a vocabulary constraint (the generating process cannot assert what has not been defined), and the verify-fix loop operating at claim-level granularity.

The contributions of this paper are:

1. **A specification** of the ProveML markup language, including syntax, verification algorithm, and grammar (Sections 3–5 and Appendix B).
2. **A threshold registry** of composable inference primitives. Domain experts define the qualitative judgments that can be verified; in regulated deployments, the prompting layer can enforce that unverifiable observations use exploratory language.
3. **An empirical evaluation** across four models (3.8B to API-scale) and two domains, with a baseline against SymGen's substitution mechanism, two ablations (context slicing, prompt language) and a markup coverage audit. Most residual failures are addressability errors rather than value mismatches, and prompt language moves the smallest model by 22 percentage points while stabilizing it (Section 7).
4. **A reference implementation** with 197 tests (plugin, grammar, paper-example, CLI, and detection checks), tested against both a generated educational dataset and real SEC EDGAR financial data (Section 10).

2 Architecture

Figure 2 contrasts the standard approach (data in, text out, no verification) with ProveML's pipeline.

2.1 Fact Store

The fact store is a flat key-value index following the Entity-Attribute-Value (EAV) pattern. Any source with records and fields can be flattened into it. The flattening is mechanical: each record becomes `type:id.field` → `value`.

Each path takes the form `entity_type:entity_id.field_name` → `value`, with optional nested sub-paths for hierarchical data (e.g., `account:901.transaction:2026-01.amount` addresses a specific transaction's amount). The verifier treats terminal values as atomic facts with canonical string representations; threshold operators may impose additional type requirements.

Examples (flat fields and nested sub-paths):

```
account:901.holder                          -> "Acme Corp"
```

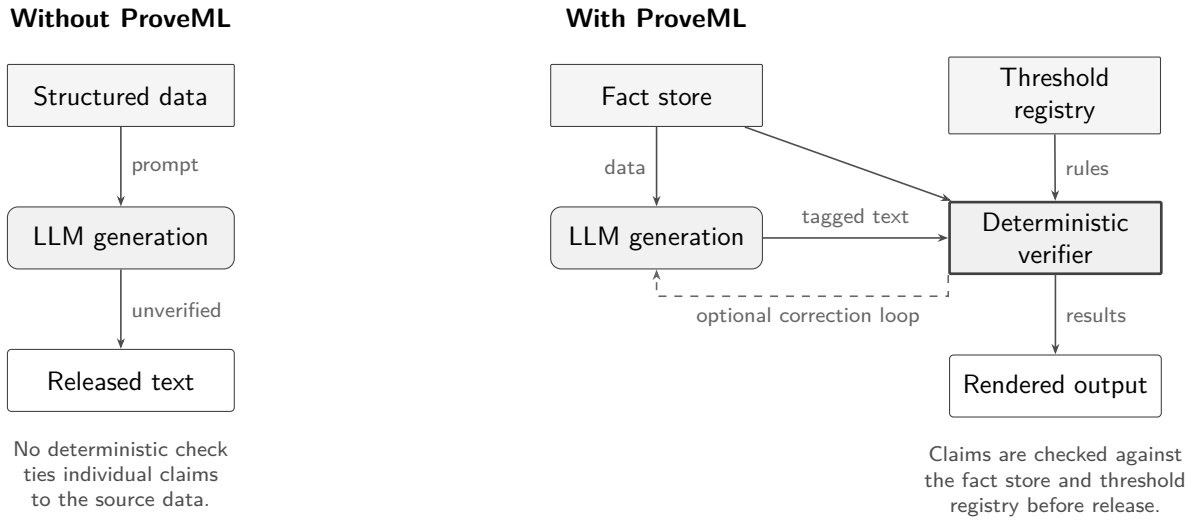

Figure 2: Without ProveML (left): data goes in, text comes out, no verification. With ProveML (right): every claim is checked against the fact store and threshold registry, with an optional correction loop. Rounded boxes denote generative components; squared boxes denote data stores and deterministic processing.

```
account:901.balance                          -> -12400
account:901.balance._unit                    -> "EUR"
account:901.overdue                          -> 3
account:901.transaction:2026-01.amount       -> 5200
account:901.transaction:2026-01.amount._unit -> "EUR"
```

ProveML separates two verification modes. Direct fact references use exact string equality against the store's canonical representation: if the store canonicalizes `18.50` to `18.5`, the claim must say `18.5`. Threshold predicates use typed comparison (numeric, set membership, etc.). The fact store is the canonicalization point: any consistent flattening scheme works, provided the same representation is used for both storage and generation.

Units. Fact store entries may carry an optional unit via a companion key:

```
account:901.balance        -> -12400
account:901.balance._unit  -> "EUR"
patient:308.glucose        -> 142
patient:308.glucose._unit  -> "mg/dL"
```

Units are modeled structurally in the fact store via companion metadata (e.g., `balance` and `balance._unit`), but direct fact references verify the canonical surface form shown to the reader (e.g., `%[balance]{-12400 EUR}`). When no unit is present, value-only comparison applies. Threshold definitions may declare an expected unit; if the fact store's unit for that field does not match, verification fails. The grammar is unchanged: unit enforcement is a property of the data model, not the markup syntax.

This follows iXBRL's principle that numeric facts should be self-describing, while keeping ProveML's verification mechanism minimal. In regulated deployments, unit strings may need to resolve to canonical identifiers from standardized vocabularies (e.g., UCUM for clinical units, ISO 4217 for currencies); this is a natural extension but beyond the scope of the present specification.

Data sources. This specification defines verification against a single flat store. The generating model verifies against whatever the deployer places in the store; source selection is a deployment decision. Multiple stores, explicit source provenance per fact, or a source prefix on entity paths are natural extensions but beyond the current scope.

ProveML does not verify the fact store itself. The system's guarantee is "consistent with the fact store", not "true". If the source data is wrong, verified claims will be wrong. The fact store is the single point of trust.

Immutability: Verification is always relative to a particular fact-store state. During a verification session, the fact store must remain immutable. If the underlying data changes, previously verified texts are not assumed to remain valid and must be re-verified. Deployments that require audit trails can bind a snapshot identifier (hash or timestamp) to each verification result; the reference implementation supports this as an optional parameter. In the evaluation, the fact store is loaded once per session and not modified, so immutability holds trivially without explicit snapshot binding.

2.2 Threshold Registry

The threshold registry draws on the design patterns of clinical reference ranges (HL7 FHIR (HL7 International, 2023)), JSON Schema validation, and monitoring alert systems (Grafana, Datadog). The common insight across these systems: domain experts think in *named ranges with semantic labels*, not in operator expressions. A clinician defines "normal glucose = 70–100 mg/dL", not "glucose >= 70 AND glucose <= 100".

Each threshold definition has a name, a field it applies to, a predicate (one of the operators in Table 1), an optional unit constraint, a human-readable label, and a source for audit. Ordering predicates (`lt`, `lte`, `gt`, `gte`, `between`, `diff_gt`) compare numerically, with `between` half-open ($low \leq v < high$, so adjacent ranges never overlap); `eq` and `neq` compare canonical string forms and therefore also apply to categorical fields; `in` tests set membership. If a unit is declared, the verifier checks it against the field's companion `._unit` value in the fact store.

Operator	Display label	Example
Core (numeric comparison)		
`lt`	is less than	`price lt 10`
`lte`	is at most	`rating lte 3`
`gt`	is greater than	`stock gt 0`
`gte`	is at least	`score gte 75`
`eq`	equals	`status eq "active"`
`neq`	is not	`status neq "archived"`
`between`	is between	`score between 25 50`
Extended		
`in`	is one of	`category in {A,B,C}`
`is_null`	is missing	`value is_null`
`diff_gt`	exceeds by more than	`_scoreDiff diff_gt 20`

Table 1: Threshold operator vocabulary. Core operators cover numeric comparison and ranges. Extended operators handle categorical data, missing values, and cross-entity comparison.

Examples of threshold definitions:

```
IS_LOW:        score lt 25         "critically low"
IS_MODERATE:   score between 25 50 "moderate"
IS_ACTIVE:     status in {A,B,C}  "active"
IS_MISSING:    value is_null       "no data"
```

```
MUCH_HIGHER:   _scoreDiff diff_gt 20 "much higher"
IS_NEGATIVE:   balance lt 0 [EUR]  "negative balance"
```

These definitions are illustrative; the reference implementation ships 20 domain-specific thresholds under its own names (e.g., `IS_LOW_PASS`, `IS_MUCH_HIGHER`). Threshold labels are field-specific, not global semantics: a range labeled "moderate" for one variable need not have the same meaning for another. The *source* field in each definition records where the threshold was defined, supporting audit.

Design rationale. The operator set is deliberately small. Logical composition (`AND`, `OR`, `NOT`) is handled at the inference level, not the threshold level, keeping individual thresholds simple enough for non-technical domain experts to define and audit. Complex patterns emerge from combining simple thresholds:

```
?[low: IS_LOW]{critically low score}
?[missing: IS_MISSING]{no recent data}
?[risk: @low AND @missing]{low score with missing data}
```

Cross-entity comparison (`diff_gt`). Consistent with the principle that the verifier only compares and never computes, cross-entity differences are materialized in the fact store by the deterministic data layer, as meta-fields on the entity being compared (e.g., `region:EU._salesDiff` holding the difference against a reference entity). The `diff_gt` operator then bounds that value:

```
?[gap: MUCH_HIGHER(region:EU._salesDiff)]{much higher}
```

verifies iff the materialized difference exceeds the registered bound. With an explicit path the operand is unambiguous (the path overrides the threshold's declared field); without one, the threshold's field resolves against the current entity context like any other threshold. The comparison is signed: the materialized difference must exceed the bound, not merely differ by it in absolute value. Keeping arithmetic in the data layer rather than the verifier means every operand of every comparison is itself an addressable, auditable fact.

Registry as vocabulary constraint. Qualitative wording is where overreach actually happens: Isch and Jennings (2026) measure causal overreach and rhetorical confidence in LLM summaries relative to their sources, finding the drift is rhetorical rather than factual — exactly the surface a value-only verifier leaves unguarded. The AI may only reference thresholds from the registry: it cannot invent a comparison value, a bound, or a direction, and an unregistered label is an error rather than a claim. What the registry constrains is therefore the *predicate* — which test runs, against which field, at which bound — and not the words the model wraps around it. A model that writes `?[rate: IS_PASSING]{a catastrophic failure}` against a passing rate produces a construct that verifies, because `IS_PASSING` holds; the display text is presentation, and the verifier does not read it. Binding wording to predicate would mean the verifier judging natural language, which is exactly the model-in-the-loop dependency the design removes. The registry narrows the space of judgments a model can make and makes each of them checkable; keeping the wording of a verified judgment honest is a review question, and Section 11.5 lists adversarial generation as out of scope.

3 Syntax

ProveML extends Markdown with three constructs, using characters (`@`, `%`, `?`) that do not conflict with standard Markdown syntax. The syntax draws on conventions from popular Markdown

extensions: bracketed spans (Pandoc, Djot), pipe-separated parameters (MediaWiki, Obsidian), and quoted metadata in references (Markdown link titles). The choice of Markdown is deliberate: LLMs produce it fluently and existing renderers pass ProveML constructs through unchanged.

A complete example before the syntax details:

```
(* Simple form: entity, then facts follow *)
@[sensor:42]{Sensor X} reads %[value]{29.99} with %[stock]{150} in stock.

(* Scoped form: facts inside entity braces *)
@[sensor:42 "Sensor X"]{reads %[value]{29.99}, %[stock]{150} in stock}

(* Nested scopes: outer entity with multiple inner entities *)
@[facility:7 "Plant North"]{hosts
  @[sensor:42 "Sensor X"]{reading %[value]{29.99}} and
  @[sensor:43 "Sensor Y"]{reading %[value]{18.5}},
  with %[sensorCount]{12} sensors total}
```

The simple form uses linear carry-forward; the scoped form uses lexical binding. In the nested example, each `value` binds to its enclosing sensor, while `sensorCount` binds to `facility:7` (outer scope). The verifier maintains a scope stack, so binding is always deterministic.

3.1 Entity Reference

Two forms, fully backward compatible:

```
@[entity_type:entity_id]{display_name}
@[entity_type:entity_id "verified_name"]{prose with facts}
```

Simple form `@[sensor:42]{Sensor X}`: the brace content serves as both the display text and the verified name. The verifier checks `fact_store`[`sensor:42.name`] = `"Sensor X"`. If the name does not match, verification fails.

Scoped form `@[sensor:42 "Sensor X"]{prose...}`: the quoted string is verified against the fact store; the braces contain prose where fact references bind explicitly to this entity. The scoped form is needed when carry-forward would give the wrong binding: multi-entity comparisons, nested entities, or languages where the subject follows the predicate:

```
@[sensor:42 "Sensor X"]{costs %[price]{29.99}} vs
@[sensor:43 "Sensor Y"]{costs %[price]{18.5}}
```

Only ProveML constructs (`@[...]`, `%[...]`, `?[...]`) inside the braces are verified; surrounding prose is not. A scoped entity does not confer verification status on its prose. It is up to the rendering engine to visually distinguish verified constructs from unverified prose, and to indicate which entity each fact binds to (for example, through color coding or scope indicators).

3.2 Fact Reference

```
%[field_name]{value}
```

A verifiable factual claim. Binding follows two rules:

1. **Lexical scope**: a fact inside `@[entity "name"]{...}` braces binds to that entity.
2. **Linear fallback**: a fact outside any scoped entity braces binds to the entity context in force at that point: the last simple-form entity seen at the current scope depth. Closing a

scope restores the context that was in force when the scope opened (a simple entity inside a scope does not leak out of it).

Rule 1 takes priority. The simple case (`@[e]{Name} ... %[f]{v}`) works as before. The scoped form handles non-English word order, tables, and interleaved comparisons without introducing explicit paths on facts; binding is always structural, never controlled by the LLM.

Verification: The verifier checks that `fact_store`$[\textit{bound\_entity.field}] = \textit{value}$.

Four outcomes: *verified* (match), *mismatch* (value differs), *unverifiable* (field not found), *no context* (no entity precedes the fact). Section 4 names the implementation's finer-grained statuses; Section 6 maps them onto rendering states.

3.3 Inference Reference

```
?[label: condition]{display_text}
```

A verifiable logical observation. Conditions reference the threshold registry or combine prior inference labels.

Conditions can be a single threshold name (e.g., `IS_LOW`), a threshold with an explicit path (e.g., `MUCH_HIGHER(region:EU._salesDiff)` for cross-entity comparison against a materialized difference), or a reference to a previously evaluated label (`@label`). These primitives compose with `AND`, `OR`, and `NOT`, evaluated with standard precedence (NOT binds tightest, then AND, then OR). The full EBNF grammar for all ProveML constructs is given in Appendix B; surrounding Markdown structure is delegated to the host parser.

4 Verification Algorithm

Figure 3 illustrates the decision logic for each ProveML construct.

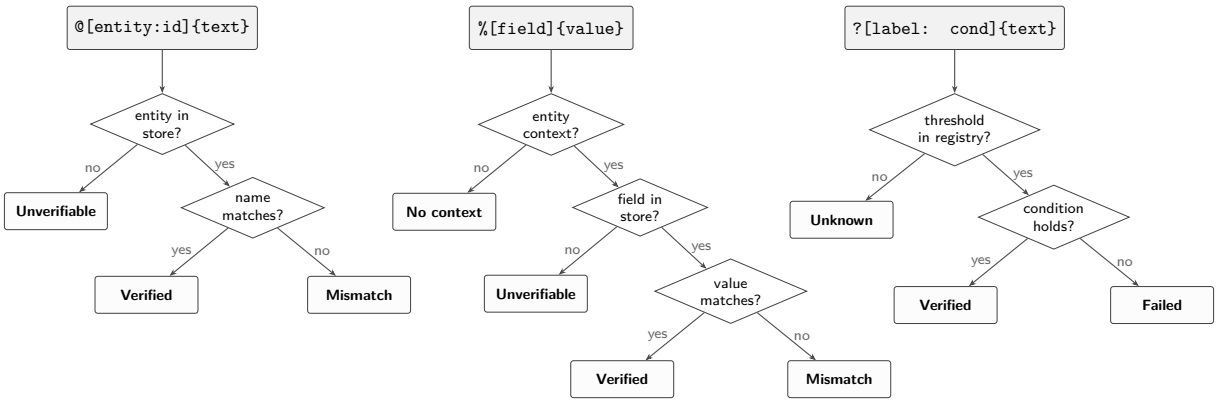

Figure 3: Verification decision flow for each ProveML construct type. Every path terminates in a deterministic outcome with no probabilistic judgment.

VerifyEntity returns Verified, EntityNotFound, or NameMismatch. VerifyFact resolves the current entity path, appends optional `._unit` metadata to the expected surface form, and compares by string equality; absence of context or fields yields NoContext or FieldNotFound. Evaluate handles registered thresholds, label references, and boolean composition. In the implementation, V.add$(\cdot)$ updates both the total claim count and the verified count when the returned result is verified.

Algorithm 1 ProveML Verification

Require: ProveML document D, fact store F, threshold registry R, optional snapshot s
Ensure: Verification result V

initialize V, *labels* $\leftarrow \emptyset$, *entityStack* $\leftarrow []$, *currentEntity* $\leftarrow$ `null`
if s is provided **then**
    V.*snapshot* $\leftarrow s$
end if
for each token t in TOKENIZE(D) **do**
    if t.*type* = `entity` **then**
        $p \leftarrow t$.*entityType*:t.*entityId*
        V.add(VERIFYENTITY(t, p, F))
        update *currentEntity* and *entityStack* from t ▷ scoped or carry-forward
    else if t.*type* = `entity_close` **then**
        restore *currentEntity* from *entityStack* if possible
    else if t.*type* = `fact` **then**
        V.add(VERIFYFACT(t, *currentEntity*, F))
    else if t.*type* = `inference` **then**
        $r \leftarrow$ EVALUATE(t.*condition*, *currentEntity*, F, R, *labels*)
        labels[t.*label*] $\leftarrow r$
        V.add(r)
    end if
end for
return V

Verification is a single linear pass over the constructs with hash-map lookups against the fact store. Condition evaluation is bounded structural recursion over a finite expression — no unbounded search, no quantifiers, no model inference — so the verifier always terminates with a deterministic result.

Comparison semantics. Fact verification uses **string equality**: the claimed value and the fact store value are both converted to their string representations and compared exactly. This means `%[price]{29.99}` matches a fact store value of `29.99` but *not* `29.990` or `30`. Threshold evaluation uses **numeric comparison** for the ordering operators (`lt`, `lte`, `gt`, `gte`, `between`, `diff_gt`): both operands are parsed as numbers before applying the operator. `eq` and `neq` compare canonical string forms and so also apply to categorical fields; `in` tests string membership; `is_null` tests presence. Implementations must define a canonical string form for fact store values (the reference implementation uses JavaScript's `String()` coercion, which drops trailing zeros: `18.50` becomes `"18.5"`). This canonicalization is implementation-defined; a Python or Rust implementation may produce different representations for the same numeric value. Portability requires that the fact store and verifier share the same canonicalization rule, and that the rule is documented per deployment. A non-numeric value under an ordering operator is a type error, not a false condition: the evaluation is invalid and the inference is reported as unverifiable. Coercing it to `NaN` and calling the condition false would let `NOT` turn a type error into a verified claim.

5 Verification Guarantee

The verification guarantee is straightforward: if the verifier reports VERIFIED for a claim, the claimed value equals the corresponding value in the fact store (for entities and facts) or the

threshold condition holds (for inferences). Each construct type reduces to a single deterministic check — string equality for entities and facts, arithmetic comparison for thresholds — with no external dependencies. The connectives use three-valued (Kleene) semantics: a condition that cannot be resolved (an unregistered threshold, a missing operand, an unresolved label reference) is UNKNOWN rather than false, true wins over unknown under `OR`, false wins under `AND`, and `NOT` propagates unknown instead of negating it. The asymmetry is deliberate: under a two-valued reading, `NOT UNREGISTERED` would verify — a claim proven by a threshold nobody defined. Unknown surfaces as UNVERIFIABLE in the rendering, distinct from a condition that resolved and was false. This simplicity is intentional. The complexity of ProveML lives in the *design* (what to verify, how to constrain the AI), not in the verification logic itself.

Assumptions. This guarantee assumes:

1. The fact store faithfully represents the source data at snapshot time.
2. The threshold registry contains meaningful thresholds defined by domain experts.
3. The fact store is immutable during verification (snapshot semantics).

Entity scoping. Fact binding uses lexical scope: a fact inside a scoped entity's braces binds to that entity. A fact outside any scoped braces binds to the entity context in force at that point; closing a scope restores the context that held at scope entry. Nested scoped entities resolve inward.

6 Rendering

ProveML specifies four verification states. The rendering must visually distinguish them; the choice of colors and styles is left to the implementation.

State	Indicator	Meaning
Verified fact	✓	Claimed value matches fact store
Verified inference	✓	Threshold condition holds
Mismatch	×	Claimed value differs from fact store
Unverifiable	?	Field or entity not found

Table 2: ProveML rendering states. The implementation's finer-grained statuses map onto these four: name mismatches and failed inference conditions render as Mismatch; entity-not-found, field-not-found, missing context, and unregistered thresholds render as Unverifiable. Visual rendering (colors, underline styles) is implementation-defined.

Three reading modes support progressive disclosure: *Read* (minimal indicators), *Verify* (full status colors with tooltips), and *Audit* (proof paths visible inline). Hovering over any entity or fact highlights all claims bound to the same entity, making the scope binding visible without inspecting markup. Figure 4 shows Verify mode on text with planted errors and Figure 5 the same text in Audit mode; Figure 1 in the introduction shows error rendering. All three use a small demonstration store of SEC EDGAR financial data plus one fictional entity (Acme Corp) for the planted errors; a live version is available at `demo/paper-screenshots-real.html` in the repository.

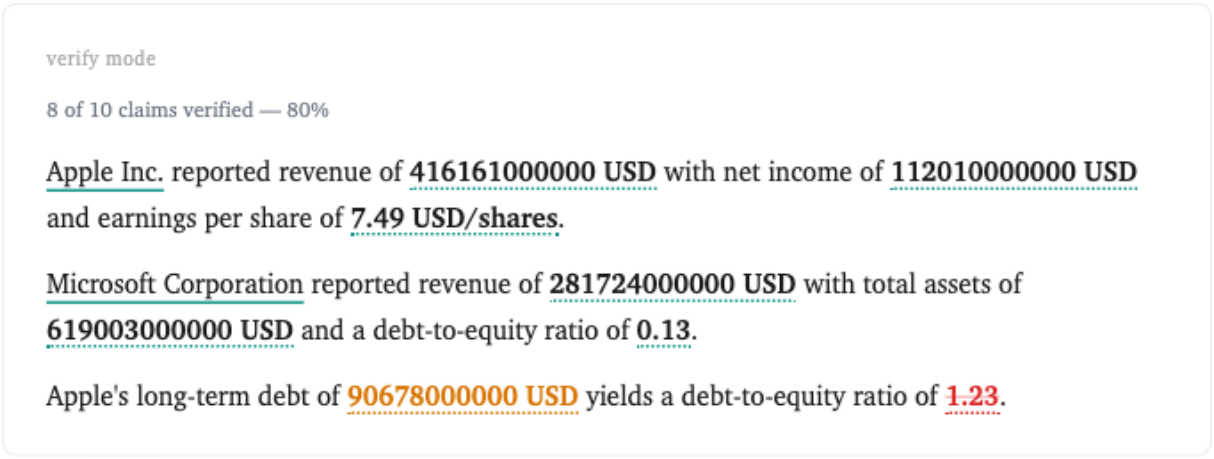

Figure 4: Verify mode on SEC EDGAR financial data. Teal dotted underlines mark verified entities and facts. Two deliberately planted errors are flagged without inspecting any markup: one claim fails to resolve in the demo fact store (amber) and the ratio built on it mismatches (red) — 8 of 10 claims verify, and the reader's attention goes straight to the other two.

7 Evaluation

The evaluation runs in five parts: a four-model study on an educational benchmark (Section 7.1), its cross-domain counterpart on SEC EDGAR filings (Section 7.2), two ablations that separate prompt language and context selection from the rest (Section 7.3), a baseline against SymGen's substitution mechanism (Section 7.4), and an audit of how much of each response is bound at all (Section 7.5). A conformance test of the verifier itself — 20 planted errors, all detected — is in Appendix A. Every run reported below is preserved in the artifact repository, and every table regenerates from those files.

The educational dataset. Two benchmarks are used below. The first is generated, in the shape of a curriculum-aligned assessment platform in Flemish secondary education (Moortgat and Deconinck, 2026): 741 pupils across 95 class offerings, each pupil carrying a mastery level per curriculum goal, plus the aggregates such a platform reports. Every record is drawn rather than transformed. Each goal is given a difficulty and each pupil a latent ability, an outcome per goal is sampled from the two, and the aggregates are computed from those outcomes; names are combined at random from lists of common Flemish given names and surnames, and the school is fictional. Nothing is derived from any real pupil, which matters here because a de-identified extract — names replaced, results kept — would still be personal data under the GDPR and would make the dataset unpublishable alongside the paper.[2]

The generator ships with the artifacts (`data/generate-education-benchmark.mjs`), is seeded, and reproduces the dataset exactly; `--verify` checks a generated dataset against a source for carried-over records. Twelve class labels and five pupil names are pinned, because the benchmark prompts ask about them by name, and class sizes are nudged so that questions about small classes have a non-empty answer; the figures behind those handles are generated like all the others. The second benchmark is built from real SEC EDGAR filings and is described in Section 7.2.

[2]We state this from experience: an earlier draft used exactly such a de-identified extract under the label "synthetic". It was withdrawn on 31 July 2026, replaced by the generated dataset described here, and every educational result in this paper was re-measured on the replacement; the artifact repository documents the withdrawal. The finance runs never touched it.

Figure 5: Audit mode on the same text as Figure 4: the exact fact store path checked for each claim is visible inline (e.g., `company:aapl.revenue`, `company:msft.debtToEquity`), exposing that the flagged long-term debt claim is bound to the wrong entity.

7.1 Four Models on an Educational Benchmark

Setup. We ran 28 prompts across 7 categories (lookup, aggregation, comparison, threshold, overview, student, unsupported) on four models: Phi-3 Mini (3.8B), Qwen 2.5 3B, Claude Haiku (API), and Qwen 2.5 7B. Each model was evaluated 3 times; we report the mean across runs. Each query received a context-sliced data payload containing only the entities relevant to that prompt (slicing is driven by the benchmark definition, which lists the entity references each prompt needs; in a production deployment, this would be handled by a retrieval or routing layer). Up to 3 correction loops were allowed. For each model we report five metrics:

- *First-pass*: verification rate before any correction loop.
- *After loop*: verification rate after up to 3 correction passes.
- *Converged*: number of queries where every claim verified. This distinguishes models that are mostly right on every query from models that are perfect on some and broken on others.
- *Addr.*: residual addressability errors (entity-not-found, name-mismatch, field-not-found) in non-converged queries.
- *Value*: residual value errors (correct reference, wrong number) in non-converged queries.

All local models ran via Ollama on an M2 Air (16GB) in Ollama's default quantization, with each backend's default decoding settings (temperature not pinned) — so the Phi-3 instability reported below is a property of this deployment configuration under default sampling, not necessarily of the weights; API snapshots are those current at evaluation time, as recorded in the run artifacts (finance March 2026; education, language-ablation and baseline runs July–August 2026). Education prompts are in Dutch, matching the dataset's origin; the finance benchmark (Section 7.2) uses English prompts. Because this difference co-varies with the benchmark contrast, we ablate it directly: Section 7.3 reports the same educational benchmark with all 28 prompts translated to English, holding data, slicing, system prompt, and loop budget fixed. Verification rate is a per-query macro-average (verified/total claims per query, a response with no markup scoring 0%), averaged over queries and then over runs; claim-weighted pooled rates can differ, and producing no verifiable markup is counted as task failure by design. Four ed-

ucation prompts and one finance prompt are "unsupported" (the data cannot answer them); they are scored with the same metric, which penalizes abstention without markup and rewards tangential verifiable claims — a dedicated abstention metric is future work. An empty answer is scored as 0% rather than dropped, since producing nothing is the extreme case of producing no markup; Phi-3 returned five such answers across its Dutch runs and none of the other models returned any. Only a call the harness kills at the wall clock is excluded from the denominator, because there no answer exists to score; one English Phi-3 call was lost this way, and no other call was lost among the runs whose harness records timeouts (the earlier finance runs predate that bookkeeping). Table 3 reports the results.

Model	First-pass	After loop	Conv. (mean)	Addr.	Value
Phi-3 Mini (3.8B)	38% ± 33.7	38% ± 34.3	7.3/28	12 (60%)	2 (10%)
Claude Haiku	88% ± 2.5	90% ± 2.5	19.7/28	18 (82%)	3 (14%)
Qwen 2.5 3B	89% ± 1.2	90% ± 1.5	20.3/28	14 (78%)	4 (22%)
Qwen 2.5 7B	89% ± 4.9	89% ± 4.9	23/28	6 (75%)	2 (25%)

Table 3: Four-model comparison (mean ± sample sd, n=3 runs). Addr. = addressability errors (residual, non-converged queries). Value = wrong number. Phi-3 also had context errors (missing entity binding). All models used sliced context, few-shot example, and up to 3 correction loops.

What the spread looks like. The four models fall into two groups, and the line between them is stability rather than rate.

1. **Phi-3 (3.8B)** is unstable across runs: 38% ± 33.7, with the three runs landing at 65%, 48% and 0%. The third run produced no verifiable markup at all and returned four empty answers. Reporting a mean for this model is close to meaningless — the honest summary is that it sometimes works and sometimes collapses entirely, and nothing in the protocol predicts which. Addressability and context errors dominate what it does produce, and the loop adds +0.6pp.

2. **Qwen 3B, Haiku and Qwen 7B** sit together at 88–89% first-pass verification and stay there. At three runs each these three are not separable: their per-run values are 88/88/90, 85/88/90 and 83/91/92, so the ranges overlap almost entirely and the ordering in Table 3 carries no signal. We report the means for completeness and draw no conclusion from the differences between them. They differ in how much they say rather than how reliably: Qwen 7B converges on the most queries (23 of 28) while producing the fewest claims (≈100 per run against Haiku's ≈172), and that convergence gap is on the order of one to one-and-a-half between-run standard deviations, so a shorter answer with less to get wrong is the more parsimonious reading.

What is *not* visible is a clean ordering by model size or capability. Haiku, an API model, sits between two local Qwen checkpoints, and the 7B model does not beat the 3B one on rate. Above the smallest model, the mechanism appears to be roughly indifferent to which model runs it.

Error classes. Across the three stable models, residual failures are dominated by addressability errors — 75–82% of the residue — rather than value mismatches. We give these as proportions rather than percentages on purpose: they are means over three runs of between six and eighteen errors each, and a percentage point would imply a precision those counts cannot carry. This suggests that ProveML's main benefit is not only catching wrong numbers, but exposing incorrect or malformed links between generated claims and the underlying data. Some

addressability errors cascade: a single wrong entity ID can produce multiple downstream field-not-found errors, so the counts overstate how many distinct mistakes lie behind them. Two caveats: models differ in claim volume (education, final claims per run: Qwen 7B ≈100, Qwen 3B ≈166, Haiku ≈172), so convergence counts are not directly comparable across models; and the class counts are means over runs with small per-run error totals, which for Phi-3 average over one run that produced nothing.

Unsupported prompts do not induce abstention. On the prompts whose answers the data cannot support, abstention was the rare exception rather than the rule: across the Dutch education runs, Qwen 3B and Haiku produced verifiable-but-tangential claims on every unsupported prompt and Qwen 7B on 10 of 12, at means of 3.8, 2.5, and 8.1 claims per response, 73–94% of them verified. On the finance benchmark all three did so on every run. Phi-3 produced fewer such claims and verified far fewer of them (2.3 per response, 25% verified), which reflects its general markup failure rather than abstention. Claim-level verification is thus orthogonal to abstention: every individual claim can be consistent with the store while the response as a whole answers a question the data cannot answer. This is the concrete case for the exploratory-language rule (Section 11.3) and for a dedicated abstention metric.

Context selection as a first-class concern. A preserved full-context ablation (one run per local model; `fullctx` artifacts in the repository) makes this failure mode concrete: given the full dataset as context instead of a slice, all three local models — including Qwen 7B, which converges on the most queries under slicing — produced *zero* ProveML constructs on every one of the 28 queries, with no call lost to a timeout. The failure is not silence: those same responses contain 62, 158 and 205 numeric tokens in plain prose (0% markup coverage by the measure of Section 7.5). Full context does not stop these models from reporting numbers — it stops them from reporting numbers that can be checked. With sliced context, the 3B model achieved 89% first-pass verification, stable across runs (± 1.2). This is not a benchmark convenience: context selection (retrieval, filtering, query routing) is standard in production deployments, and ProveML should be evaluated inside that architecture. Note the flip side: our slicing is driven by the benchmark's own entity lists, i.e. an oracle retriever, so the headline rates assume retrieval has already succeeded; measuring markup quality under an imperfect retriever is future work. The finding is that verification and context selection work together: context selection brings models above the capability threshold, and verification ensures the claims they produce are correct. Neither alone is sufficient.

What the markup buys is the measurement itself. Bare prose can be checked after the fact — by regular expressions against the store, by a model asked to judge, or by generating from templates so that values are correct by construction — but none of those yields a per-claim verdict with zero variance and a path back to the record that produced it. The verify-fix loop and the error classification above exist because every claim is addressable; without that, the same responses give a noisy estimate rather than a count.

7.2 Cross-Domain Evaluation on SEC EDGAR Data

Setup. To test domain-agnostic applicability, we built a second benchmark from real SEC EDGAR XBRL filings: FY2025 10-K data for Apple Inc. (CIK 0000320193, filed 2025-10-31) and Microsoft Corporation (CIK 0000789019, filed 2025-07-30). The fact store contains 11 financial fields per company (revenue, assets, liabilities, net income, EPS, cash, equity, shares outstanding, long-term debt, debt-to-equity ratio, current ratio), unit-annotated where a unit applies (USD, USD/shares, shares; the debt-to-equity ratio is dimensionless). We designed 10 prompts across 5 categories (lookup, comparison, threshold, overview, unsupported). Each

model was evaluated 3 times with the same pipeline as Section 7.1: sliced context, few-shot example, up to 3 correction loops. Table 4 reports the results.

Model	First-pass	After loop	Conv. (mean)
Phi-3 Mini (3.8B)	92% ± 2.5	92% ± 2.5	8.3/10
Claude Haiku	92% ± 2.6	97% ± 3.0	8.7/10
Qwen 2.5 3B	93% ± 5.8	93% ± 5.8	9.3/10
Qwen 2.5 7B	99% ± 2.3	99% ± 2.3	9.7/10

Table 4: Finance benchmark (mean ± sample sd, n=3 runs). SEC EDGAR FY2025 data, 10 prompts, 2 entities, unit-annotated. All models used sliced context, few-shot example, and up to 3 correction loops.

Benchmark structure, not model size. On this compact, structurally clean benchmark (2 entities, 11 fields each, English prompts), all four models entered the verifiable boundary reliably — including Phi-3 Mini, which scored 92% ± 2.5 here against 38% ± 33.7 on the educational benchmark (95 class offerings, 741 students). The instability seen there disappears: Phi-3's three finance runs land within 5 points of one another, where its three educational runs span 65 points. Even the weakest model produces stable, high-quality markup when the schema is compact, the entities are few, and the prompts are in English. Section 7.3 shows how much of that is the language.

Loop benefit tracks the model, not the benchmark. On this benchmark the loop added nothing for three of four models; only Haiku improved (+5pp). Across all three benchmark variants Haiku is the only model that improved consistently (+2.6pp Dutch, +3.3pp English, +5.0pp finance), while Qwen 7B gained at most +1pp anywhere and Phi-3 gained only where it had verifiable markup to repair (+3.0pp on English prompts, +0.6pp on Dutch). Each of those deltas sits within about one between-run standard deviation, and the loop cannot go negative by construction (a correction is accepted only if it improves verification), so at n=3 this is a consistent direction rather than an established effect. Read as an observation, the loop is selective in a specific sense: it helps models that already produce well-formed markup with repairable addressing errors, and it cannot rescue a model that produces no markup at all. Where the loop adds nothing, ProveML's value is compliance and auditability rather than correction.

Implications. These results do not show that finance is "easier" than education; they show that the capability threshold depends strongly on benchmark structure. The two benchmarks differ in more than entity count — prompt language (Dutch vs. English), domain, prompt mix, and schema size all co-vary. Section 7.3 ablates the largest of these factors and finds that language accounts for a substantial part of the gap for the smallest model and almost none of it for the other three. The same architecture, verifier, and grammar produced very different results across domains, which supports ProveML's domain-agnostic design: the mechanism transfers; the deployment challenge is context management.

7.3 Language Ablation: Separating Prompt Language from Benchmark Structure

Setup. The two benchmarks of Sections 7.1 and 7.2 differ in prompt language as well as in structure, so the contrast between them cannot by itself locate the cause. We therefore translated all 28 educational prompts to English (identical prompt ids, entity references, context

slices, system prompt, and loop budget; only the natural-language question changed) and re-ran all four models 3 times. The translated benchmark and its run artifacts are in the repository (`benchmarks/proveml-pilot-en.v1.json`, `en` run files).

	First-pass verification		Markup coverage	
Model	**Dutch**	**English**	**Dutch**	**English**
Phi-3 Mini (3.8B)	38% ± 33.7	60% ± 8.7	60.7%	64.8%
Claude Haiku	88% ± 2.5	89% ± 2.9	91.9%	87.7%
Qwen 2.5 3B	89% ± 1.2	89% ± 5.8	72.5%	87.9%
Qwen 2.5 7B	89% ± 4.9	87% ± 3.5	60.5%	69.7%

Table 5: Language ablation on the educational benchmark: same data, same slices, same prompts in Dutch and in English (mean ± sample sd, n=3 runs each). The language effect is confined to the smallest model.

The language effect is a small-model effect, and mostly a stability effect. Translating the prompts moves Phi-3 Mini from 38% ± 33.7 to 60% ± 8.7 first-pass verification (+22pp) and from 7.3 to 13.3 of 28 queries converged. The mean shift is the less interesting half. What the translation really does is stop the model from falling over: its Dutch runs are 65%, 48% and 0%, its English runs 70%, 58% and 53%. On Dutch prompts the model has a mode in which it produces nothing at all, and on English prompts it does not. A mean of 38% describes neither state.

For the three larger models the effect is negligible and does not point one way: Haiku +1pp, Qwen 3B ±0, Qwen 7B −2pp, all well inside run-to-run variation. Coverage moves more than rate does for the two Qwen models (3B 72.5% → 87.9%, 7B 60.5% → 69.7%) while Haiku moves the other way (91.9% → 87.7%), which is a reminder that the two metrics are not measuring the same thing. Both language conditions were stored under the same 2,000-character cap, and it bound rarely and roughly evenly (Phi-3 3 of 84 Dutch responses, 7 of 83 English; no other model reached it), so the coverage comparison is not an artifact of truncation.

What this revises. Two conclusions follow. First, the ordering between models is not an artifact of language: even on English prompts Phi-3 remains far below the other three (60% against 87–89%), so translating the request does not turn a 3.8B model into a reliable one. Second, within the educational benchmark, language moves this model roughly 22pp; how much of the remaining gap to finance (38% against 92%) is benchmark structure cannot be separated further without the missing cell, a Dutch-language compact benchmark, which we did not run — but a claim that context complexity alone explains the gap would already be wrong, and so would a claim that language alone does. Non-English deployment is thus a first-class variable for small models under ProveML, and it shows up less as a lower score than as a model that intermittently produces nothing: the same schema, verifier, and thresholds yield markedly different reliability depending on the language the request arrives in.

7.4 Baseline: Substitution versus Verification

Setup. SymGen (Torroba Hennigen et al., 2024) is the closest published mechanism, and it takes the opposite strategy: the model emits Jinja-like `{{ field }}` references into the conditioning data and a parser substitutes each one, so the model never states a value itself. Unresolvable references render as `undefined` — the default the SymGen paper itself suggests, though rendering is a deployment choice and their own interface instead highlights provenance. We reimplemented its Direct strategy against the same fact stores, prompts, context slices and

models used above (3 runs per model per benchmark; the Indirect generate-then-rewrite variant is not reimplemented). One asymmetry remains: ProveML's delivered responses passed through up to three correction loops, while SymGen generated in a single pass — its mechanism has no error signal to loop on, but a resolve-and-retry variant is conceivable and was not built. The Caught column is first-pass and unaffected; the coverage comparison inherits the asymmetry, though for three of the four models the loop changes almost nothing.

We baseline against SymGen rather than the nearer neighbour, Proof-Carrying Numbers (Solatorio, 2025), for a specific reason: SymGen's mechanism is fully specified in its paper down to the delimiter, so a faithful reimplementation is possible from the published description alone. PCN shares three of ProveML's four properties (deterministic, structured, inline) and differs mainly in the threshold layer, which makes it the more informative comparison in principle; we did not attempt it because reimplementing it from the preprint would have meant guessing at parts of the protocol, and a baseline built on guesses is worse than none. A direct comparison remains the most useful experiment someone could run next.

Because the designs answer different questions, a single accuracy number would be misleading. Substitution cannot produce a wrong value for a reference that resolves — SymGen's value-error count is zero by construction, which is a genuine property of the design. What is comparable is how much of the output each mechanism binds to the data, and what each can detect.

	SymGen		**ProveML**	
Benchmark	**Binding**	**Resolved**	**Coverage**	**Caught**
Finance (SEC EDGAR)	83.6%	94.9%	85.9%	43
Education	75.7%	78.0%	71.4%	212

Table 6: SymGen reimplementation against ProveML on identical models, prompts and data (4 models × 3 runs each). Binding/Coverage: share of numeric tokens in the delivered text that is tied to the data. The two are not the same measure: an unresolved SymGen reference renders as `undefined` and so leaves the text entirely, while an unverifiable ProveML claim stays in it as a marked number. Counting unresolved references as attempted bindings instead gives SymGen 85.0% and 80.1% (averaged over models, like the main columns). Resolved is pooled over references, Binding and Coverage are averaged over models. Caught: claims ProveML flagged on the first pass.

On education, substitution binds more and resolves less. On the educational benchmark SymGen binds a larger share of the numeric output than ProveML (75.7% against 71.4%, or 80.1% counting unresolved references as attempts), which is unsurprising: emitting a placeholder is easier than transcribing a value correctly. On the compact finance benchmark the two are level (83.6% against 85.9%, or 85.0% against 85.9% under the other count) and we draw no conclusion from that row. But 306 of its 1,392 educational references (22%) address a path that does not exist, and each of those renders as the literal string `undefined` in the delivered text. Forty-one percent of the educational responses contain at least one. On the compact finance benchmark the same figures are 5% of references and 10% of responses.

Both mechanisms fail on addressability; they fail differently. ProveML is not immune to this: on the same benchmark, 155 of its 1,528 first-pass claims (10%) carry an addressability error, and addressability dominates its residual failures (75–82%, Section 7.1). Neither approach prevents a model from choosing the wrong path when there are 95 entities to choose between, though substitution does so at roughly twice the rate here. The difference is what the reader receives: SymGen's failure is a hole in the sentence, ProveML's is a claim carrying a status

and the value that was expected instead. ProveML flagged 212 claims on the first pass of the educational benchmark, 40 of them wrong values — a class SymGen cannot produce, and equally cannot report.

What substitution cannot do at all. Two capabilities are structural rather than statistical. SymGen cannot check a document it did not generate: with no claimed values in the text there is nothing to compare, so auditing existing reports (Section A) has no analogue. And it has no mechanism for qualitative claims: "critically low" is not a field in any data structure, so a substituting model must either omit such language or emit it unbound. ProveML's threshold registry exists precisely for that surface.

7.5 Markup Coverage Audit

Motivation. The verification rate counts only claims *inside* ProveML markup. A generator could achieve a high rate by marking up very little and leaving the rest of its numbers in plain prose. Verification rate therefore needs a complementary metric: *markup coverage*, the fraction of numeric content that sits inside a verifiable construct at all. Santillana (2026) reach the same conclusion from the evaluation side, showing that reference-free faithfulness metrics measure only precision over stated claims and therefore reward abstention; they pair precision with recall against a completely derived oracle. Markup coverage is the structural analogue of that recall term: the fact store plays the role of the complete oracle, and the question becomes how much of the response is bound to it.

Method. We ran a deterministic token audit over the stored final responses of the education and finance studies (Sections 7.1 and 7.2; all four models, 3 runs each). Coverage is defined as $marked/(marked + unmarked)$, where *marked* counts `%[field]{value}` constructs containing a digit, and *unmarked* counts standalone numeric tokens remaining after stripping all ProveML constructs from the response. Excluded from the unmarked count: list markers, calendar and fiscal years, digits embedded in identifiers (e.g., class codes like `6ZW`), and code spans. The audit script is included in the research repository (`experiments/coverage-audit.mjs`). Coverage is a single figure aggregated over the three runs of each cell, measured on the delivered (post-loop) responses; for three of the four models the loop changes almost nothing, and we report no between-run spread for it. Stored responses are truncated at 2,000 characters; the cap binds rarely and only for Phi-3 (3 of 84 Dutch education responses, 7 of 83 English, 2 of 30 finance; no other model reached it), so no cell is measured on materially truncated text. Table 7 reports coverage per model and benchmark.

Model	Education	Finance
Phi-3 Mini (3.8B)	60.7%	84.6%
Claude Haiku	91.9%	86.7%
Qwen 2.5 3B	72.5%	91.9%
Qwen 2.5 7B	60.5%	80.2%

Table 7: Markup coverage: fraction of standalone numeric tokens in final responses that appear inside `%[...]{...}` markup, aggregated over 3 runs per model per benchmark.

Findings. Coverage is high but never complete, and it does not track verification rate. The clearest demonstration is that the three stable models are indistinguishable on rate — 88–89%, with overlapping run ranges — and more than 30 percentage points apart on coverage: Haiku marks 91.9% of its numeric tokens, Qwen 7B 60.5%. Two models can therefore look equally

trustworthy under the headline metric while one leaves nearly two fifths of its numbers as unverifiable prose. Coverage also responds to prompt language, but not in a consistent direction (Table 5): translating the prompts raises coverage for the three local models and lowers it for Haiku, so it is not a proxy for competence either. The two metrics must be read together; verification rate alone overstates the verified fraction of a report's numeric content. Inspection of the unmarked tokens shows three clusters: (1) **query-echoed thresholds** ("classes below 60%"), which should be inference constructs referencing the registry; (2) **derived values** with no addressable path in the fact store — counts, differences, and benchmark constants (the comparison bound "1.0" in a liquidity judgment, or a count of classes the model tallied itself) — which the current specification cannot express as facts; and (3) **store-addressable values left in plain prose**. Token-level classification between these clusters is inherently ambiguous (a bare "5" can be a count, an echoed threshold, or a store value), so we do not rank them: what the audit establishes is the size of the unmarked residue, not its composition. Clusters 1 and 3 are prompt-compliance failures that coverage measurement makes visible and a stricter system prompt can target; cluster 2 motivates future work on derived-value claims.

8 Deployment

The evaluation measures whether models can produce verifiable markup. This section reports what it costs to run and what a deployment has to build, because both are load-bearing for anyone deciding whether to adopt the mechanism, and neither follows from the tables above.

Cost of verification: negligible. Verifying 515 claims across 84 stored responses against a 1,672-key fact store takes 100 ms on a laptop — 1.2 ms per response, under 0.2 ms per claim. Verification is a hash lookup and a comparison per claim, so it scales with the number of claims, not with the size of the store. Against generation latency (26–52 s per query for the local models in our runs) this is free, which is what allows verification to sit inside the generation loop rather than after it.

Cost of generation: markup and retries. Two overheads are real. Marked-up responses ran 52–60% longer in characters than the same text with the markup stripped, for the three models that produce markup reliably (Phi-3, which often produces little, sits at 15%). That is a direct output-token cost. The correction loop adds a second: across the educational runs, 74% of queries converged with no correction pass at all and 25% exhausted all three, with almost nothing in between. The bimodality matters more than the mean — budgeting should assume a query either costs one generation or four, not the 1.8 the average suggests. A deployment that caps the loop at one pass keeps most of the benefit, since Haiku's gains were +2.6 to +5.0 percentage points across all three benchmark variants and the first pass carries most of that.

What a deployment has to build. The fact store is the work. Flattening records into `type:id.field` is mechanical once the entities are decided, but deciding them is a modelling exercise: a normalized database with joins does not announce what counts as one addressable entity, and that choice determines which claims can be bound at all. Derived values are the recurring case — a count, a difference, a ratio the report computes on the fly — and the rule is that arithmetic belongs in the data layer, materialized as an addressable fact, rather than in the verifier (Section 4). A threshold registry is optional at the entity-and-fact level and small when present: twenty entries covered both benchmarks here.

The retrieval assumption, stated plainly. Our headline rates assume retrieval has already succeeded: context slices come from the benchmark's own entity lists, an oracle retriever. The full-context ablation shows what happens without slicing — all three local models produced zero constructs across all 28 queries while still emitting 62 to 205 numeric tokens in plain prose (Section 7.1). So context selection is not a convenience of the setup, it is a precondition of the mechanism, and its quality in a real deployment is the largest variable this paper does not measure. Treat the reported rates as an upper bound conditional on retrieval, and budget for the retrieval work accordingly.

Minimal integration. The reference implementation is an npm package with no runtime dependencies:

```
import { verifyProveml } from 'proveml/verify';

const store = { 'student:42.name': 'Alex', 'student:42.passRate': 5 };
const text  = '@[student:42]{Alex} scored %[passRate]{5}%.';

const { verified, total, errors } = verifyProveml(text, store);
// -> 2 / 2, no errors
```

A deployment adds three things to an existing LLM call: the store, a system prompt that asks for the markup, and the verifier on the response. The verify-fix loop and the rendering layer are optional on top.

9 Related Work

We organize related work by what each system checks, how it checks, and where in the pipeline it operates.

Post-hoc fact-checking (AI in the loop). FActScore (Min et al., 2023), SAFE (Wei et al., 2024), FacTool (Chern et al., 2023), and OpenFactCheck (COLING 2025) decompose generated text into atomic claims and verify each using LLMs or web search. ClaimVer (EMNLP 2024 Findings) adds explainability through knowledge graphs. SelfCheckGPT (Manakul et al., 2023) detects hallucinations via sampling consistency without external knowledge. RefChecker (Hu et al., 2024) extracts claim triplets for reference-based checking. All use AI in the verification loop, making verification itself probabilistic. Akhter et al. (2026) show that decomposition helps only when the evidence is granular and aligned per sub-claim — a property ProveML's fact store has by construction rather than by retrieval. GAVEL (Xu et al., 2026) moves partway toward a binding contract: debating agents must state atomic subclaims bound to explicit evidence units, and a scrutinizer validates cited identifiers and quoted spans deterministically — but the debate that produces the subclaims is model-driven, and the evidence units are text spans rather than addressable data paths. Pronesti et al. (2026) make the same architectural argument we do, in the reasoning-step setting: they replace neural judges over chain-of-thought with rule-based verifiers precisely because model judges are opaque and reward-hackable. FinGround (Guo et al., 2026) is the closest 2026 system: it decomposes financial answers into typed atomic claims, recomputes arithmetic ones against structured tables, and rewrites unsupported claims with table-cell citations. Its verification is hybrid (decomposition and typing are model-based) and post-hoc on free text, and it emits citations to source locations rather than binding a claimed value to an addressable path. ProveML's verification is deterministic end to end.

Attribution (where, not whether). Anthropic's Citations API, OpenAI's response annotations, and Google's Vertex AI Grounding (Google, 2024) link generated text to source documents or passages. Google's grounding API additionally returns support scores and claim-to-source metadata, though verification remains probabilistic. RARR (Gao et al., 2023a) and WebGPT (Nakano et al., 2021) attach URLs. LAQuer (ACL 2025) and "Attribute First, then Generate" (ACL 2024) localize attributions to fine-grained source spans. These systems track *where* a claim came from, not *whether the claimed value is correct*. Verification, if any, is against unstructured text. GenProve (Wei et al., 2026) learns to emit provenance triples alongside the answer, and reports a sharp gap between surface quotation, which models handle, and inference-backed provenance, which they do not — a split that maps onto which claims ProveML can bind deterministically at all. The AIS framework (Rashkin et al., 2023) gives this field its shared evaluation vocabulary — whether a statement is attributable to identified sources, as judged by humans — and ALCE (Gao et al., 2023b) benchmarks citation generation directly, scoring whether generated statements are supported by the passages they cite. For the wider landscape, Schreieder et al. (2026) survey 134 papers and 300 metrics on evidence-based generation. Onweller et al. (2026) quantify the consequence: parsing inline citations out of LLM-generated Markdown is deterministic and reliable, but checking what those citations support is not, and even frontier models' citations frequently fail that check.

Grounding guardrails (confidence scoring). The academic line runs from NLI-based inconsistency detection (SummaC, Laban et al., 2022) through learned alignment metrics (AlignScore, Zha et al., 2023) to distilled claim-checkers (MiniCheck, Tang et al., 2024); industrially, Amazon Bedrock Guardrails, Azure AI Groundedness, Vectara HHEM, Patronus Lynx, Galileo, Ragas, and FaithJudge (EMNLP 2025 Industry Track) produce probabilistic confidence scores for response faithfulness. These are post-generation filters, not inline markup. None performs deterministic claim-level verification against structured data. Amazon Bedrock's Automated Reasoning checks (2025) are adjacent to ProveML but not equivalent: Bedrock validates model outputs against formalized policy constraints, whereas ProveML places claim-level verification markup directly inside the generated text, binding individual claims to addressable data paths and registered threshold predicates. The overlap is strongest at the inference/policy layer: Bedrock tells you whether a response complies; ProveML lets you ship a response whose verified parts are explicitly marked, auditable, and enforceable at the level of individual claims.

Schema validation (format, not facts). Instructor, Outlines, LMQL, Guidance, and Pydantic AI constrain LLM output *structure*: valid JSON, correct types, matching schemas. They ensure the AI produces the right shape; ProveML ensures the AI produces the right values. The two are complementary in a way this paper does not exploit: since most residual failures are malformed or misdirected bindings rather than wrong numbers, constraining decoding to well-formed ProveML with only existing paths would remove much of what we measure as addressability error. The machinery for it exists: grammar-constrained decoding masks inadmissible tokens during generation (PICARD, Scholak et al., 2021) and can enforce semantic constraints beyond syntax (Synchromesh, Poesia et al., 2022), so constraining generation to well-formed ProveML with only existing paths is an application of a known technique, not a research program. We did not test it, and it is the most obvious next step for anyone building on the mechanism.

Data-grounded generation. Fact verification against evidence has a canonical benchmark lineage — FEVER for textual sources (Thorne et al., 2018), TabFact for tables (Chen et al., 2020), FEVEROUS for both at once (Aly et al., 2021) — in which a trained model judges whether evidence supports a claim; ProveML sits outside that lineage by making the judgment a lookup

rather than a model. The data-to-text community (Parikh et al., 2020) and StructFact (ACL 2025 Findings) benchmark faithful generation from structured data. ClaimDB (Theologitis et al., 2026) benchmarks fact verification over large structured databases, where reading the evidence breaks down and verification shifts to executable programs; it also reports that models struggle to abstain when the data cannot decide a claim, which matches our finding in Section 7.1. FinVerBench (Panda, 2026) verifies statements against SEC XBRL filings and finds measured performance shifts with numeric rendering (rounded vs. unrounded) — an external measurement of the canonicalization sensitivity we discuss in Section 11.5. Kamu Data's Oracle-Augmented Generation (Kamu Data, 2025) delegates computation to a deterministic query engine with cryptographic provenance. These approaches verify at the query level; ProveML verifies at the claim level within natural language text.

Inline tagging. The idea of annotating human-readable text so that machines can resolve parts of it is older than any of this. RDFa (W3C, 2015) adds attributes to HTML that bind spans of prose to entities and properties in a structured vocabulary, resolvable without reading the prose — the same shape of idea, developed for documents people wrote themselves. What ProveML adds to that lineage is not the binding but the verdict: RDFa says what a span refers to, and assumes the author meant it; ProveML asks whether the claim survives comparison with the record. iXBRL (XBRL International, 2013) embeds machine-readable tags in human-readable financial reports, the closest structural analog among document standards. LLMON (Hind et al., 2026) carries structure and semantic metadata across the LLM interface, primarily to separate instructions from data; it structures the interface, whereas ProveML makes the output checkable. Proof-Carrying Numbers (Solatorio, 2025) is the closest recent neighbour: the LLM emits claim-bound tokens tying numeric spans to structured claims, verified deterministically in the renderer with explicit status marks and tolerance policies; it lacks entity scoping and a composable threshold registry. SymGen (Torroba Hennigen et al., 2024) embeds symbolic references in AI-generated text that are resolved against table data. SymGen's strategy is *error prevention*: a parser substitutes placeholders with data values, so the AI never independently claims a value. ProveML's strategy is *error detection*: the AI claims values directly, and a verifier checks them (Figure 6). SymGen cannot detect mismatches or audit existing text. ProveML adds entity scoping, a threshold registry, and composable inference chains. Section 7.4 reports a reimplementation of its Direct strategy on our benchmarks.

Provenance standards. W3C PROV-O (Lebo et al., 2013) and C2PA (Coalition for Content Provenance and Authenticity, 2022) provide document-level provenance metadata. Proof-Carrying Code (Necula, 1997) embeds machine-checkable safety proofs in executable code. ProveML borrows the principle that the producer embeds checkable evidence; the mechanisms differ entirely.

10 Reference Implementation

A reference implementation is available as a markdown-it plugin (JavaScript):

- ProveML plugin (parser + renderer with lexical scope stack): 550 lines of JavaScript
- Threshold registry: 20 thresholds, exercising the full operator vocabulary of Table 1
- Test suite: 197 tests (115 plugin, 33 grammar, 15 paper examples, 14 CLI, 20 detection)
- Convergence harness: 28 prompts, 4 models, 3 repeats (Section 7.1)

Approach	Category	Determ.	Structured	Inline	Inference
FActScore, SAFE, FacTool	Fact-checking	–	–	–	–
Anthropic/OpenAI/Google	Attribution	–	–	Partial	–
Bedrock/Azure/Lynx/HHEM	Guardrails	–	–	–	–
Instructor/Outlines/LMQL	Schema	✓	–	–	–
ClaimDB, Kamu OAG	Data-grounded	✓	✓	–	–
iXBRL*	Inline tagging	✓	✓	✓	–
SymGen	Inline tagging	✓	✓	✓	–
PCN	Inline tagging	✓	✓	✓	–
FinGround	Hybrid post-hoc	Partial	✓	–	–
ProveML	**Inline tagging**	✓	✓	✓	✓

Table 8: Comparison across representative verification approaches. Determ. = deterministic. Structured = against structured data. Inline = claim-level markup in text. Inference = composable threshold checks within natural language text. The Bedrock/Azure/Lynx/HHEM row reflects the predominant probabilistic grounding and guardrail mode; Amazon Bedrock's separate Automated Reasoning feature adds formal policy checks but not inline claim markup. FinGround recomputes arithmetic claims deterministically but decomposes and types them with a model, hence Partial. *XBRL's formula linkbase provides document-level calculation and assertion checks; the iXBRL specification itself does not include claim-scoped inference.

Fact store construction. The server loads a JSON data file and flattens it into a key-value store at startup: each entity's fields become `type:id.field` → `value` entries. Optional `._unit` companion keys are added where units are declared. The flattening is a single pass over the source data with no external dependencies.

Verify-fix loop. The generation pipeline calls the LLM, passes the response to the standalone verifier, and inspects the result. If errors remain and the loop budget is not exhausted, a correction prompt is constructed containing the specific verification errors and the original data context. The corrected response is accepted only if it improves verification without dropping claims.

Rendering integration. The markdown-it plugin implements the same verification semantics against the same fact store and threshold registry as the standalone verifier (which is the normative component), then emits HTML spans with `data-*` attributes carrying the verification status, entity path, and error details. A CSS layer maps these attributes to visual indicators. The plugin and standalone verifier share the same fact store and threshold registry but are independently usable. That sameness is a maintained invariant rather than a given: an audit during the preparation of this paper found the plugin had drifted to two-valued condition evaluation, which rendered the negation of an unregistered threshold as verified; the suite now pins the two implementations to each other on exactly these cases.

Trust adapters. The implementation also carries a layer this specification deliberately does not cover: a fact store can be wrapped in a *trust adapter* that attaches a trust status (`verified`, `unverified`, `expired`, `revoked`, `error`) and a proof reference to each resolved value — for stores backed by signed sources such as verifiable credentials — and verification results merge these worst-first, so one revoked input marks every claim it touched. Claim verification and source trust stay orthogonal: a claim can match a value whose signature has expired, and the result carries both facts separately. The adapter interface ships with the package; its semantics are implementation-documented rather than specified here, and nothing in Sections 3–5 depends on it.

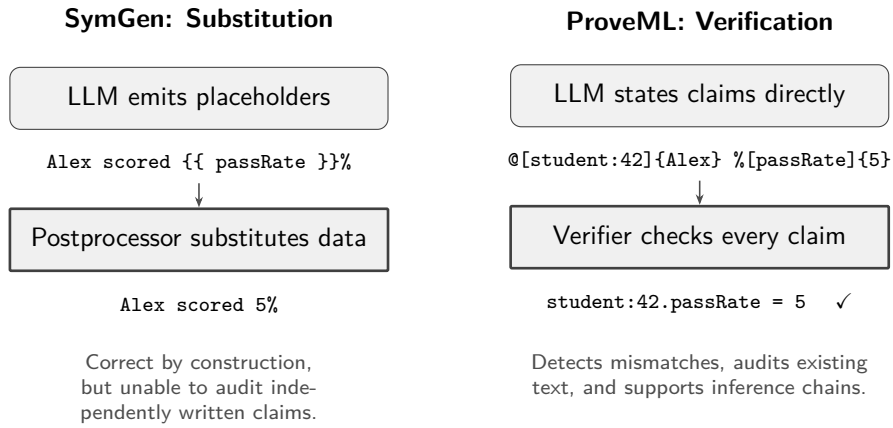

Figure 6: Architectural comparison. SymGen prevents errors by substituting placeholders with data values. ProveML detects errors by verifying independently claimed values against the fact store.

Tested using a generated dataset in the shape of a secondary school assessment platform. Every record is drawn from the model described in Section 7, none is derived from a real pupil, and the school does not exist. The reference implementation is available as an npm package (`proveml`) with source at https://github.com/ShaneDeconinck/proveml. The paper, benchmarks, experiment results, and reproducibility guide are at https://github.com/ShaneDeconinck/proveml-research.

11 Discussion

11.1 What the Boundary Is For

Although ProveML can improve output quality through verify-fix loops, its more durable contribution is the boundary it draws: a line inside a document separating claims that carry a machine-checkable status and a path back to a record from text that carries neither.

The verify-fix loop is a consequence of this design. Because verification is deterministic and instant, errors can be fed back to the LLM with specific messages ("student:42.passRate: claimed 7, actual 5"), enabling targeted correction at claim-level granularity. But the loop is most useful for models that already produce well-formed markup. Without adequate context scaffolding, models may produce too little verifiable markup for the loop to help. For the most capable models, verification serves primarily as an audit trail. The loop's value is in the transition zone where models are capable enough to produce markup but not reliable enough to get it right on the first pass.

The deeper value is not the loop but the boundary itself. With ProveML markup, every marked claim carries a verification status and an addressable path, so a reader — or an auditor months later — can ask of any given number where it came from and get an answer that does not depend on a model. Prose without that structure can still be checked, but only in the aggregate and only by someone willing to re-derive it. What changes is the unit of accountability: from "was this report right" to "was this claim right, and against which record".

11.2 Incentive-Aligned Generation

A key design goal of ProveML is that the markup language should be *easy for LLMs to use correctly*. This is not incidental; it creates a productive alignment between the model's generation objective and the system's verification objective.

Maximizing verifiable surface area. A deployment's system prompt sets how much of the output it asks to be marked: at minimum every entity in `@[entity:id]{name}` and every number in `%[field]{value}`, and where qualitative language matters, every judgment in `?[label:THRESHOLD]{text}`. The more a prompt asks for, the more of the response arrives as verifiable constructs and the smaller the residue of unchecked prose, which is what the coverage audit in Section 7.5 measures. The prompt used throughout our evaluation asks only for the first two, so the numbers reported here describe entity and fact markup and say nothing about how well models produce inference constructs; that is a gap in the evaluation, not a property of the language, and Section 11.5 states it as such.

Why LLMs are well-suited to this. Three properties of modern LLMs make ProveML generation natural rather than burdensome:

1. **Markdown fluency**: LLMs are extensively trained on Markdown and produce it natively. ProveML's constructs (`@[...]`, `%[...]`, `?[...]`) use characters that do not conflict with Markdown syntax, making them easy to interleave with natural language.
2. **In-context learning**: A few examples in the system prompt are sufficient for LLMs to adopt the ProveML pattern in practice. Prompt examples alone were enough for every model we tested to adopt the pattern, with no fine-tuning (Section 7.1).
3. **Structured data access**: When the fact store is provided in the context window, the model has direct access to the exact values it should reference. The task reduces to *selecting and formatting* rather than *recalling from parametric memory*, precisely the setting where LLMs are most reliable.

Model-agnostic by design. ProveML deliberately avoids requiring fine-tuned or specialized models. The system prompt, fact store, threshold registry, and verify-fix loop together form a *skill* that any general-purpose LLM can execute. As foundation models improve, ProveML benefits automatically: fewer first-pass errors, fewer correction iterations. The right abstraction is not "train the model to be more verifiable" but "give the agent the tools to prove its claims."

11.3 The Claim Boundary: Domain Experts Define What Can Be Stated

Not every sentence in AI-generated text needs to be a verified claim. The goal is not to annotate 100% of the prose, but to give domain experts a precise way to mark which claims should be checked. This creates a clear boundary between two modes of AI output:

1. **Verified claims**: facts, entity references, and inferences that resolve against the fact store and threshold registry. These are rendered with verification indicators and can be trusted to be consistent with the data.
2. **Plain prose**: text outside ProveML markup. It remains ordinary prose with no verification status and should not be treated as a checked claim.

The threshold registry defines which qualitative judgments *can* be verified. Claims inside the registry get checked deterministically. Text outside the markup remains unchecked prose. Deployments may prompt for broader use of markup, but ProveML itself does not scan or reject plain prose outside the verified layer.

In practice, the system prompt can encourage authors to keep consequential judgments inside the verified layer:

```
RULES:
- Every qualitative word MUST use ?[label: THRESHOLD]{text}
- If no threshold exists for your observation, write it as:
  "This may warrant further investigation" (NOT "This is X")
```

If the AI writes an unregistered threshold name inside a ProveML construct, the verifier returns an error. Otherwise, plain prose remains unchecked.

Progressive adoption. Adoption is not all-or-nothing. At Level 1 (entity references and fact values only), ProveML already detects wrong numbers. At Level 2, the threshold registry constrains qualitative language. Deploying requires only addressable data and, optionally, threshold definitions — a flat configuration file per domain. The verifier, grammar, and verify-fix loop are unchanged across domains. The design also extends naturally to settings with machine-verifiable attestations (verifiable credentials, SSI systems, blockchain-derived state), though these applications are forward-looking and not evaluated in this paper.

11.4 Scope and Privacy

ProveML verifies what the AI *marks up*: every entity name, every number, and every thresholded qualitative judgment inside a ProveML construct. Text outside the markup remains unchecked prose. The system's guarantee is "consistent with the fact store." The fact store is the single point of trust.

Privacy by design. Because entity references are structural (`@[student:42]{...}`) and the display text is separate from the identifier the verifier resolves, a deployment can hand the model pseudonymous identifiers and have the rendering layer substitute names at display time: the verification status travels with the markup, the identifying data stays behind. This is a property the syntax makes available, not one it enforces, and our evaluation does not use it — the benchmark contexts carry generated pupil names and the models write them out. Whether a pseudonymous configuration costs anything in markup quality is untested.

11.5 Limitations

- **Consistency, not truth.** ProveML verifies that claims match the fact store, not that the fact store is correct. If the source data is wrong, verified claims will be wrong.
- **No unit conversion.** Units are supported as nullable metadata with exact string matching. Semantic unit equivalence (e.g., mg/dL vs mmol/L) and standardized vocabularies (UCUM, ISO 4217) are deferred to future work.
- **Evaluation scope.** The evaluation covers two domains (generated education, real SEC finance) across 38 prompts, 4 models, and 3 repeats. Whether error patterns generalize to larger schemas, more domains, or frontier models is not exhaustively established.
- **Inference constructs are unmeasured.** The system prompt used throughout the evaluation asks for entity and fact markup only. No run produced an inference construct, so every rate we report is a rate over entities and facts, and the threshold registry — the part

of the design that constrains qualitative language — is exercised by the specification, the test suite and the worked examples, but not by the four-model study. How reliably small models emit `?[label: THRESHOLD]{text}`, and how often they reach for a threshold that fits the sentence rather than the data, is an open question this paper does not answer.

- **Rounded prose.** Exact string equality forbids rounded or reformatted values: a report cannot say "$391 billion" when the store holds `391035000000`. Verified prose must state canonical values, which constrains natural financial and journalistic phrasing. Tolerance-based matching and display-time formatting of verified values are future work.
- **Prose outside markup.** ProveML verifies what is inside markup constructs. Text outside the markup is not checked; detecting unverified claim-bearing prose is outside the scope of the current specification. Section 7.5 quantifies the numeric portion of this gap: coverage ranges from 60.5% to 91.9% depending on model, benchmark, and prompt language.
- **Malformed input.** The tokenizer silently skips constructs with unmatched brackets or braces. Robustness to diverse malformed markup deserves further stress-testing.
- **Adversarial generation.** ProveML is designed against models that make mistakes, not against a model trying to mislead. A generator that wanted to could choose a threshold that holds and wrap misleading words around it, mark only the claims that verify and leave the damaging ones as prose, or select a true fact that misrepresents the record it came from. Each of these survives verification because the verifier reads paths and values, not intent. We neither measure nor defend against this.
- **Group claims.** Inferences evaluate against a single entity context. A claim like "5TW, 4B, and 6WE all have low coverage" cannot be verified as one inference; it must be decomposed into one inference per entity.

11.6 Future Work

Several extensions are deferred from the current specification: (1) standardized unit vocabularies (UCUM, ISO 4217) for semantic unit equivalence rather than string matching; (2) multiple fact stores with explicit source provenance or a source prefix on entity paths; (3) cross-implementation portability of canonicalization rules; (4) derived-value claims (counts, differences, ratios computed from store values), which the current specification cannot express as facts at all (Section 7.5); (5) tolerance-based matching and display-time formatting for rounded prose; (6) further ablations separating context size from domain and prompt mix (prompt language is ablated in Section 7.3); and (7) evaluation on larger schemas, more domains, and frontier models, with an abstention metric for unsupported queries.

12 Conclusion

ProveML places AI-generated claims inside a verifiable boundary. Three inline constructs link every claim to an addressable path in a data source. Verification is deterministic, with no model in the loop. An optional threshold registry extends this to qualitative judgments through composable primitives.

In our evaluation across four models (3.8B to API-scale, 3 runs each), we find that the capability threshold depends strongly on the shape of the request rather than on model size: on an educational benchmark with 95 class offerings and 741 students, Phi-3 scored 38% ± 33.7 with individual runs at 65%, 48% and 0%; on a compact SEC EDGAR finance benchmark (2 entities), the same model scored 92% ± 2.5 with all three runs within 5 points, the architecture, verifier, and grammar unchanged. Ablations locate two of the responsible factors: within the educational benchmark, prompt language moves this smallest model roughly 22 percentage

points and the larger three almost nothing, and giving any local model the full dataset instead of a slice drives markup production to zero — not to silence, but to unverifiable prose.

What LLMs provide is open-ended generation. What ProveML adds is a controllable, auditable boundary around that generation. For humans, the intended benefit is visual triage: verified claims carry a status, and attention shifts to what is unverified. We have not measured whether this reduces human verification effort, and the evidence from adjacent systems is mixed: Torroba Hennigen et al. (2024) report their user study reduced average verification time by 20%, while Mehrotra et al. (2026) report a 21-participant study in which verification and correction effort did not differ significantly from their baseline, even though their system reduced hallucination. Which of the two ProveML's rendering resembles in practice is an open empirical question. For agent loops the benefit is more direct and does not depend on human factors: specific error feedback ("expected 142, got 143") enables targeted self-correction, which is what the verify-fix loop measures in Section 7. The combination is what neither open-ended generation nor static verification offers alone: open-endedness with determinism.

Declaration of Generative AI Use

Generative AI tools (Claude, GPT) were used during manuscript preparation for drafting, revision, literature search, and code review. All AI-generated content was reviewed, verified, and edited by the author. No generative AI was used to create or alter images or artwork. The author takes full responsibility for the content of the published article.

References

M. E. Akhter, F. Ruggeri, I. M. Bilal, R. Procter, and M. Liakata. The alignment bottleneck in decomposition-based claim verification. arXiv:2602.10380, 2026.

R. Aly, Z. Guo, M. Schlichtkrull, J. Thorne, A. Vlachos, C. Christodoulopoulos, O. Cocarascu, and A. Mittal. FEVEROUS: Fact extraction and VERification over unstructured and structured information. In *Proc. NeurIPS Datasets and Benchmarks Track*, 2021. arXiv:2106.05707.

L. Bainbridge. Ironies of automation. *Automatica*, 19(6):775–779, 1983. doi: 10.1016/0005-1098(83)90046-8.

W. Chen, H. Wang, J. Chen, Y. Zhang, H. Wang, S. Li, X. Zhou, and W. Y. Wang. TabFact: A large-scale dataset for table-based fact verification. In *Proc. ICLR*, 2020. arXiv:1909.02164.

I-C. Chern, S. Chern, S. Chen, W. Yuan, K. Feng, C. Zhou, J. He, G. Neubig, and P. Liu. FacTool: Factuality detection in generative AI – a tool augmented framework for multi-task and multi-domain scenarios. *arXiv:2307.13528*, 2023.

Coalition for Content Provenance and Authenticity. C2PA technical specification. https://c2pa.org/specifications/, 2022.

M. Coulter and G. Bensinger. Google AI chatbot Bard offers inaccurate information in company ad. Reuters, 2023.

European Parliament and Council of the European Union. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act). Official Journal of the European Union, 2024.

European Parliament and Council of the European Union. Regulation (EU) 2026/1744 of 8 july 2026 amending regulations (EU) 2024/1689, (EU) 2018/1139 and (EU) 2023/1230 as regards the simplification of the implementation of harmonised rules on artificial intelligence (Digital Omnibus on AI). Official Journal of the European Union, https://eur-lex.europa.eu/eli/reg/2026/1744/oj/eng, 2026.

S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630:625–630, 2024. doi: 10.1038/s41586-024-07421-0.

L. Gao, Z. Dai, P. Pasupat, A. Chen, A. T. Chaganty, Y. Fan, V. Zhao, N. Lao, H. Lee, D. Juan, and K. Guu. RARR: Researching and revising what language models say, using language models. In *Proc. ACL*, 2023a. doi: 10.18653/v1/2023.acl-long.910.

T. Gao, H. Yen, J. Yu, and D. Chen. Enabling large language models to generate text with citations. In *Proc. EMNLP*, pages 6465–6488, 2023b. doi: 10.18653/v1/2023.emnlp-main.398.

Gartner. More than 80% of enterprises will have used generative AI APIs or deployed generative AI-enabled applications by 2026. Press Release, 2023.

Google. Grounding with Google search. Gemini API documentation, https://ai.google.dev/gemini-api/docs/grounding, 2024.

Guardrails AI. Guardrails: Adding guardrails to large language models. https://github.com/guardrails-ai/guardrails, 2024.

D. Guo, J. Wu, and S. M. Yiu. FinGround: Detecting and grounding financial hallucinations via atomic claim verification. In *Proc. ACL Industry Track*, 2026. arXiv:2604.23588.

M. Hind, B. Shbita, B. Wu, F. Ahmed, C. DeLuca, N. Fulton, D. Cox, and D. Gutfreund. LLMON: An LLM-native markup language to leverage structure and semantics at the LLM interface. arXiv:2603.22519, 2026.

HL7 International. FHIR (fast healthcare interoperability resources), release 5, 2023. URL https://hl7.org/fhir/R5/.

Xiangkun Hu, Dongyu Ru, Lin Qiu, Qipeng Guo, Tianhang Zhang, Yang Xu, Yun Luo, Pengfei Liu, Yue Zhang, and Zheng Zhang. Knowledge-centric hallucination detection. In *Proc. EMNLP*, pages 6953–6975, 2024. doi: 10.18653/v1/2024.emnlp-main.395.

C. Isch and G. Jennings. Narrative license and model sycophancy in LLM summaries of scientific work. In *Proc. ACL*, 2026. doi: 10.18653/v1/2026.acl-long.746.

Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 2023. doi: 10.1145/3571730.

A. T. Kalai and S. S. Vempala. Calibrated language models must hallucinate. In *Proc. STOC*, 2024. arXiv:2311.14648.

A. T. Kalai, O. Nachum, S. S. Vempala, and E. Zhang. Why language models hallucinate. OpenAI, arXiv:2509.04664, 2025.

Kamu Data. Oracle-augmented generation: Connecting AI to real-time verifiable data. https://www.kamu.dev/blog/2025-01-08-oracle-augmented-generation/, 2025.

O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *Proc. ICLR*, 2024. arXiv:2310.03714.

P. Laban, T. Schnabel, P. N. Bennett, and M. A. Hearst. SummaC: Re-visiting NLI-based models for inconsistency detection in summarization. *Transactions of the Association for Computational Linguistics*, 10:163–177, 2022. doi: 10.1162/tacl_a_00453.

T. Lebo, S. Sahoo, D. McGuinness, et al. PROV-O: The PROV ontology. W3C Recommendation, https://www.w3.org/TR/prov-o/, 2013.

J. D. Lee and K. A. See. Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1):50–80, 2004. doi: 10.1518/hfes.46.1.50_30392.

Y. Liu, X. Peng, J. Cao, X. Wang, S. Deng, J. Chen, J. Yin, and X. Zhang. ToolGate: Contract-grounded and verified tool execution for LLMs. In *Findings of ACL*, 2026. doi: 10.18653/v1/2026.findings-acl.470.

V. Magesh, F. Surani, M. Dahl, M. Suzgun, C. D. Manning, and D. E. Ho. Hallucination-free? assessing the reliability of leading AI legal research tools. *Journal of Empirical Legal Studies*, 22(2):216–242, 2025. doi: 10.1111/jels.12413.

P. Manakul, A. Liusie, and M. J. F. Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proc. EMNLP*, 2023. doi: 10.18653/v1/2023.emnlp-main.557.

M. McCain, T. Millar, S. Huang, J. Eaton, K. Handa, M. Stern, et al. Measuring AI agent autonomy in practice. Anthropic Research, https://www.anthropic.com/research/measuring-agent-autonomy, 2026.

McKinsey & Company. The state of AI. https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai, 2025.

N. Mehrotra, A. Kumar, S. Gulwani, A. Radhakrishna, and A. Tiwari. TEN: Table explicitization, neurosymbolically. In *Proc. ACL Industry Track*, 2026. doi: 10.18653/v1/2026.acl-industry.138.

S. Min, K. Krishna, X. Lyu, M. Lewis, W. Yih, P. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proc. EMNLP*, 2023. doi: 10.18653/v1/2023.emnlp-main.741.

Rony Moortgat and Shane Deconinck. Designing curriculum-aligned digital assessment infrastructures: a design-based case study of Naviskore in Flemish secondary education. *Frontiers in Education*, 11, 2026. doi: 10.3389/feduc.2026.1856724.

R. Nakano et al. WebGPT: Browser-assisted question-answering with human feedback. *arXiv:2112.09332*, 2021.

G. C. Necula. Proof-carrying code. In *Proc. POPL*, pages 106–119, 1997. doi: 10.1145/263699.263712.

H. Onweller, E. Lumer, A. Huber, P. Ramchandani, V. K. Subbiah, and C. Feld. Cited but not verified: Parsing and evaluating source attribution in LLM deep research agents. arXiv:2605.06635, 2026.

H. Orgad, M. Toker, Z. Gekhman, R. Reichart, I. Szpektor, H. Kotek, and Y. Belinkov. LLMs know more than they show: On the intrinsic representation of LLM hallucinations. In *Proc. ICLR*, 2025. arXiv:2410.02707.

L. Ouyang et al. Training language models to follow instructions with human feedback. In *Proc. NeurIPS*, 2022. arXiv:2203.02155.

S. Panda. FinVerBench: Benchmark validity and calibration in large language model financial statement verification. arXiv:2605.29586, 2026.

N. Paoli. Deloitte was caught using AI in $290,000 report to help the Australian government crack down on welfare after a researcher flagged hallucinations. Fortune, https://fortune.com/2025/10/07/deloitte-ai-australia-government-report-hallucinations-technology-290000-refund/, 2025.

R. Parasuraman and V. Riley. Humans and automation: Use, misuse, disuse, abuse. *Human Factors*, 39(2):230–253, 1997. doi: 10.1518/001872097778543886.

A. Parikh, X. Wang, S. Gehrmann, M. Faruqui, B. Dhingra, D. Yang, and D. Das. ToTTo: A controlled table-to-text generation dataset. In *Proc. EMNLP*, 2020. doi: 10.18653/v1/2020.emnlp-main.89.

Paulo Pinheiro da Silva, Deborah L. McGuinness, and Richard Fikes. A proof markup language for Semantic Web services. *Information Systems*, 31(4–5):381–395, 2006. doi: 10.1016/j.is.2005.02.003.

G. Poesia, O. Polozov, V. Le, A. Tiwari, G. Soares, C. Meek, and S. Gulwani. Synchromesh: Reliable code generation from pre-trained language models. In *Proc. ICLR*, 2022. arXiv:2201.11227.

M. Pronesti, A. Belz, and Y. Hou. Beyond outcome verification: Verifiable process reward models for structured reasoning. In *Findings of ACL*, 2026. doi: 10.18653/v1/2026.findings-acl.1611.

H. Rashkin, V. Nikolaev, M. Lamm, L. Aroyo, M. Collins, D. Das, S. Petrov, G. S. Tomar, I. Turc, and D. Reitter. Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4):777–840, 2023. doi: 10.1162/coli_a_00486.

T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen. NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In *Proc. EMNLP System Demonstrations*, pages 431–445, 2023. doi: 10.18653/v1/2023.emnlp-demo.40.

J. S. Santillana. Precision is not faithfulness: Coverage-aware evaluation of grounded generation with a complete oracle. arXiv:2606.09376, 2026.

T. Scholak, N. Schucher, and D. Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proc. EMNLP*, pages 9895–9901, 2021. doi: 10.18653/v1/2021.emnlp-main.779.

T. Schreieder, T. Schopf, and M. Färber. Attribution, citation, and quotation: A survey of evidence-based text generation with large language models. In *Proc. ACL*, 2026. doi: 10.18653/v1/2026.acl-long.1430.

Aivin V. Solatorio. Proof-carrying numbers (PCN): A protocol for trustworthy numeric answers from LLMs via claim verification. arXiv:2509.06902, 2025.

L. Tang, P. Laban, and G. Durrett. MiniCheck: Efficient fact-checking of LLMs on grounding documents. In *Proc. EMNLP*, 2024. doi: 10.18653/v1/2024.emnlp-main.499.

M. Theologitis, P. P. S. Dammu, C. Shah, and D. Suciu. ClaimDB: A fact verification benchmark over large structured data. In *Proc. ACL*, pages 34428–34451, 2026. doi: 10.18653/v1/2026.acl-long.1589. arXiv:2601.14698.

J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal. FEVER: a large-scale dataset for fact extraction and VERification. In *Proc. NAACL-HLT*, pages 809–819, 2018. doi: 10.18653/v1/N18-1074.

L. Torroba Hennigen, S. Z. Shen, A. Nrusimha, B. Gapp, D. Sontag, and Y. Kim. Towards verifiable text generation with symbolic references. In *Proc. COLM*, 2024. arXiv:2311.09188.

U.S. District Court, S.D.N.Y. *Mata v. Avianca, Inc.* 678 F.Supp.3d 443, 2023.

W3C. RDFa 1.1 primer — third edition. W3C Working Group Note, https://www.w3.org/TR/rdfa-primer/, 2015.

J. Wei, C. Yang, X. Song, Y. Lu, N. Hu, J. Huang, D. Tran, D. Peng, R. Liu, D. Huang, C. Du, and Q. V. Le. Long-form factuality in large language models. In *Proc. NeurIPS*, 2024. arXiv:2403.18802.

J. Wei, X. Wang, Y. Liao, J. Dong, Y. Liu, C. Jia, B. Yu, and J. Zhu. GenProve: Learning to generate text with fine-grained provenance. In *Proc. ACL*, 2026. doi: 10.18653/v1/2026.acl-long.228.

XBRL International. Inline XBRL part 1: Specification 1.1. https://www.xbrl.org/specification/inlinexbrl-part1/rec-2013-11-18/inlinexbrl-part1-rec-2013-11-18.html, 2013.

R. Xu, G. Li, and V. S. Sheng. GAVEL: Evidence-contract debate with mechanized scrutiny for provenance-grounded fact-checking. In *Findings of ACL*, 2026. doi: 10.18653/v1/2026.findings-acl.1789.

Z. Xu, S. Jain, and M. Kankanhalli. Hallucination is inevitable: An innate limitation of large language models. arXiv:2401.11817, 2024.

Y. Zha, Y. Yang, R. Li, and Z. Hu. AlignScore: Evaluating factual consistency with a unified alignment function. In *Proc. ACL*, pages 11328–11348, 2023. doi: 10.18653/v1/2023.acl-long.634.

A Error Detection Rate

Setup. The critical question for ProveML is not whether the LLM generates correct output, but whether ProveML *detects* incorrect output. To test this, we injected 20 deliberate errors into otherwise valid ProveML text and measured whether the verifier caught each one.

Results. The 20 errors span six categories: wrong values (5), wrong entity (3), no context (2), wrong threshold (4), cross-entity swaps (2), and subtle errors including trailing spaces and swapped entity order (4). ProveML detected all 20, yielding a **100% error detection rate**. This is a consequence of deterministic equality checking: if the claimed value does not exactly match the fact store, the claim is flagged regardless of plausibility. The experiment validates that

the implementation behaves as specified — it is a conformance test, not a detector benchmark — and it is one-sided: it measures detection of planted errors, not false positives on correct-but-reformatted values (a canonicalization difference such as `18.50` vs. `18.5` is flagged despite numeric equality; see Limitations). Robustness to malformed LLM markup is partially addressed by the convergence experiment (Section 7.1) but deserves further stress-testing.

B ProveML Grammar (EBNF)

The following EBNF captures the ProveML constructs as implemented in the reference parser. It is intended as a starting point for formal treatment, not a normative specification; the authoritative definition is the reference implementation and test suite. Where the two diverge, the reference tokenizer is deliberately *more permissive* than this grammar — for example, it accepts identifiers beyond the character classes below and tolerates a missing space after the inference label colon — so the grammar describes the canonical surface form generators should produce, while the tokenizer accepts sloppier variants of it. One exception runs the other way: the tokenizer brace-counts entity display text, so an unbalanced open brace (admitted by the `display_text` production) causes the construct to be skipped.

```
(* ProveML constructs inside Markdown inline content *)
entity_ref_simple = "@[" , entity_path , "]{" , display_text , "}" ;
entity_ref_scoped = "@[" , entity_path , " " , DQUOTE ,
                    entity_name , DQUOTE , "]{" ,
                    scoped_content , "}" ;
entity_ref        = entity_ref_simple | entity_ref_scoped ;

fact_ref          = "%[" , field_name , "]{" , value_text , "}" ;
inference_ref     = "?[" , label , ": " , condition , "]{" ,
                    display_text , "}" ;

(* Paths and identifiers *)
entity_path       = identifier , ":" , entity_id ;
entity_id         = ( letter | digit ) ,
                    { letter | digit | "_" | "-" } ;

field_name        = field_segment , { "." , field_segment } ;
field_segment     = meta_field | path_field | identifier ;
meta_field        = "_" , identifier ;
path_field        = identifier , ":" , entity_id ;

label             = identifier ;
identifier        = letter , { letter | digit | "_" } ;

(* Text content *)
entity_name       = { char - '"' - ']' } ;
display_text      = { char - "}" } ;
scoped_content    = { char } ;   (* parsed recursively *)
value_text        = { char - "}" } ;

(* Condition grammar *)
condition         = or_expr ;
or_expr           = and_expr , { " OR " , and_expr } ;
and_expr          = unary_expr , { " AND " , unary_expr } ;
unary_expr        = [ "NOT " ] , primary ;
primary           = threshold_name
                  | threshold_name , "(" , path , ")"
                  | "@" , label ;
```

```
(* Terminals *)
threshold_name    = uppercase , { uppercase | "_" } ;
path              = entity_path , "." , field_name ;
DQUOTE            = '"' ;
letter            = "a"-"z" | "A"-"Z" ;
uppercase         = "A"-"Z" ;
digit             = "0"-"9" ;
```